
Rapport d'avancement de stage

Semaines 6 & 7

Synthese vocale par clonage de voix (XTTS)
et integration : la requete unifiee vers un JSON exploitable

Un copilote d'intelligence artificielle pour le controle aerien

Auteur Nicolas Marano
Projet Simulateur de controleur aerien (PoC, 10 semaines)
Etablissement Universite de Reims Champagne-Ardenne (URCA)
Calcul (IA) Supercalculateur ROMEO : NVIDIA GH200 (Grace-Hopper, aarch64)
Domaine Intelligence artificielle & controle du trafic aerien
Periode Semaines 6 et 7 du stage

Resume. Ces deux semaines couvrent l'achevement de la boucle de dialogue puis le debut de son integration. La semaine 6 ajoute le module de synthese vocale : un clonage de voix *zero-shot* (XTTS-v2) genere a la volee une voix distincte par pilote virtuel, suivie d'une degradation VHF (filtre de Butterworth 300–3400 Hz) qui reproduit le canal radio ; la synthese est mesuree a 3,9 s par collationnement et le taux d'erreur mots de re-transcription de la voix synthetisee est de 22 a 24 %. La semaine 7 construit une *requete unifiee* qui assemble la transcription, les regles du RAG et une situation spatiale fixe pour produire un JSON conforme decrivant l'action ; une chaine de validation deterministe garantit que ce JSON reste propre et exploitable.

Table des matières

1	Contexte et positionnement des deux semaines	1
1.1	Synthese chiffree des deux semaines	1
2	Semaine 6 : Synthese vocale par clonage de voix	1
2.1	Methode : clonage <i>zero-shot</i> avec XTTS-v2	2
2.2	Chaine de synthese	2
2.3	Generation du collationnement (phraseologie OACI)	2
2.4	Voix de reference	3
2.5	Degradation VHF et coherence de domaine	3
2.6	La boucle vocale complete	4
2.7	Resultats et mesures	5
3	Semaine 7 : Integration – la requete unifiee	5
3.1	Les trois entrees de la requete	5
3.2	La situation spatiale fixe : le graphe de secteur	5
3.3	Construction de la requete unifiee	6
3.4	Orchestration : de la transcription au JSON	7
3.5	Garantir un JSON propre et exploitable	7
3.6	Resultats et mesures	7
4	Bilan et transition	8

1 Contexte et positionnement des deux semaines

Le projet automatise le dialogue radio pilote / controleur a l'interieur d'un simulateur de trafic aerien. Une instruction parlée est transcrite, ancrée dans la phraseologie OACI, transformée en une commande validée, exécutée dans le simulateur BlueSky, puis restituée en voix de pilote synthétisée. La chaîne cible se resume ainsi :

voix pilote (VHF) → Whisper STT → NER + LLM avec RAG (OACI Doc 4444) → JSON valide
→ simulateur BlueSky (manoeuvre avion) → voix de readback synthetisee → boucle

Les semaines precedentes ont produit le module de **perception** (semaine 4, reconnaissance vocale par Whisper fine-tune) et le module d'**interpretation** par RAG (semaine 5, LLM ancre produisant un JSON valide et borne par une validation de securite). Les semaines 6 et 7 couvrent deux progressions complementaires. La semaine 6 ajoute le **dernier maillon** de la chaine, la **restitution vocale** : l'aeronef ne se contente plus d'exécuter la manoeuvre, il repond sur la frequence dans une voix qui lui est propre. La semaine 7 ouvre la phase d'**integration** : plutot que de traiter chaque entree isolement, le systeme reunit la transcription, les regles de phraseologie et une representation de l'espace aerien au sein d'une **requete unifiee**. Le projet passe ainsi de la construction de modules isoles a leur assemblage.

1.1 Synthese chiffree des deux semaines

Semaine	Indicateur	Resultat
S6	Latence de synthese (par collationnement, CPU)	3,9 s (RTF 0,82)
S6	WER de re-transcription de la voix synthetisee	22 a 24 %
S6	WER de reference sur voix humaine reelle (validation S4)	6,7 %
S6	Voix de reference clonees (pilotes / controleur)	3
S7	Entrees combinees dans la requete unifiee	3
S7	Situation spatiale fixe (fixes / segments / separation)	7 / 6 / 5 NM
S7	JSON exploitable produit (40 transcriptions reelles)	100 %
S7	Couches de validation du JSON de sortie	4

Table 1 – Vue d’ensemble des resultats obtenus durant les semaines 6 et 7.

2 Semaine 6 : Synthese vocale par clonage de voix

Objectif de la semaine

Implementer une solution de **clonage vocal** permettant de generer a la volee l’audio des differents pilotes virtuels. Chaque aeronef doit pouvoir prononcer son collationnement avec une voix qui lui est propre, sans entraînement spécifique par locuteur, et dans un rendu sonore compatible avec le canal radio VHF du simulateur.

2.1 Methode : clonage *zero-shot* avec XTTS-v2

La generation repose sur le modele XTTS-v2 (paquet `coqui-tts`). Le choix d’un clonage *zero-shot* evite tout entraînement par locuteur : le modele reproduit le timbre d’une voix a partir d’un seul extrait de reference de quelques secondes, fourni au moment de la synthese. Cette approche permet d’attribuer une voix distincte a chaque aeronef sans constituer de corpus dedie ni lancer de campagne d’entraînement supplémentaire, ce qui est adapte a la contrainte de calcul et de stockage du cluster.

Composant	Valeur	Remarque
Modele	XTTS-v2 (<code>coqui-tts</code>)	multilingue, clonage zero-shot
Strategie	clonage sans entraînement	voix clonee depuis un extrait de reference
Voix de reference	3 extraits ATCO2 (4 a 9 s)	une voix par pilote virtuel
Frequence native	24 kHz → 16 kHz	rééchantillonnage vers l’entree ASR
Langue	en	phraseologie anglaise
Materiel d’inference	CPU (noeud GH200)	contournement du bug cuFFT (GPU)
Post-traitement	degradation VHF 300–3400 Hz	rendu radio (<i>sim-to-real</i>)

Table 2 – Configuration de la synthese vocale (semaine 6).

2.2 Chaîne de synthese

La fonction `synth()` enchaîne quatre étapes déterministes après la génération XTTS : rééchantillonnage de 24 kHz vers 16 kHz (fréquence d'entrée du pipeline ASR), normalisation d'amplitude pour éviter l'écroulement, puis dégradation VHF. La sortie est un signal mono 16 kHz directement consommable par le reste de la chaîne.

texte de readback (OACI) → XTTS-v2 (clonage, voix de référence) → rééchantillonnage 24→16 kHz
→ normalisation → dégradation VHF (Butterworth 300-3400 Hz) → wav 16 kHz

Listing 1 – Cœur de la synthèse : clonage XTTS puis post-traitement (extrait de `tts_atc.py`).

```
wav = np.asarray(tts.tts(text=text, speaker_wav=ref_wav, language=language), dtype=np.float32)
wav16 = resample_poly(wav, FS, XTTS_SR).astype(np.float32) # 24k -> 16k
peak = np.max(np.abs(wav16)) + 1e-9
if peak > 1.0:
    wav16 = wav16 / peak
if vhf:
    wav16 = preprocess_waveform(wav16, training=False) # bande passante VHF 300-3400 Hz
```

2.3 Génération du collationnement (phraseologie OACI)

Le texte prononcé n'est pas l'instruction brute mais un *collationnement* conforme à la phraseologie. Le module `readback.py` convertit les ordres JSON en téléphonie standard : chiffres épelés un à un (`niner` pour 9), indicatifs en code téléphonique (`AFR` → *air france*), points de cheminement en alphabet OACI. Le sens d'un changement d'altitude (montée ou descente) est déterminé en comparant la valeur cible à l'altitude courante de l'aéronef.

Listing 2 – Exemple de collationnement généré à partir d'un ordre JSON.

```
Ordre : {callsign: AFR1234, action: ALT, value: 10000} (avion au FL130)
Texte : "descend flight level one zero zero, air france one two three four"
```

2.4 Voix de référence

Les voix de référence sont extraites du corpus ATCO2 (jeu de test) par le module `make_pilot_voices.py` : il sélectionne trois segments courts (4 à 9 s, non silencieux) qui servent de timbres à cloner. Chaque aéronef de la flotte se voit attribuer une de ces voix, de sorte que des pilotes différents sonnent différemment sur la fréquence. L'usage d'extraits radio réels comme référence contribue au réalisme du rendu.

2.5 Dégradation VHF et cohérence de domaine

La voix synthétisée par XTTS est propre (large bande). Pour la rendre crédible sur une fréquence aéronautique, la chaîne applique la dégradation VHF déjà validée en semaine 2 : un filtre passe-bande de Butterworth d'ordre 6, limite à la bande 300-3400 Hz (figure 1). Cette étape a un second intérêt, au-delà du réalisme : elle place la voix synthétisée dans le *même domaine acoustique* que les données sur lesquelles le module de reconnaissance vocale a été spécialisé. La re-transcription du collationnement reste ainsi cohérente avec le reste de la chaîne (réduction du fossé *sim-to-real*). La figure 2 illustre l'effet de ce filtrage sur un signal de parole.

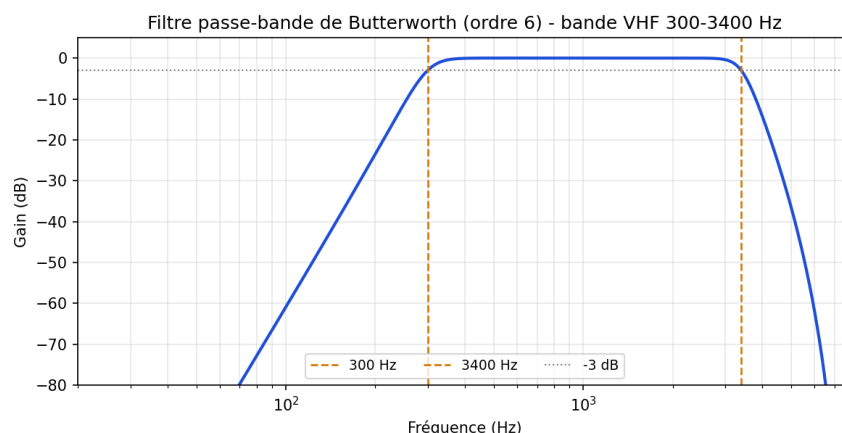


Figure 1 – Reponse en frequence du filtre passe-bande de Butterworth (ordre 6) utilise pour la degradation VHF. La bande utile est limitee a 300–3400 Hz, comme une radio aeronautique.

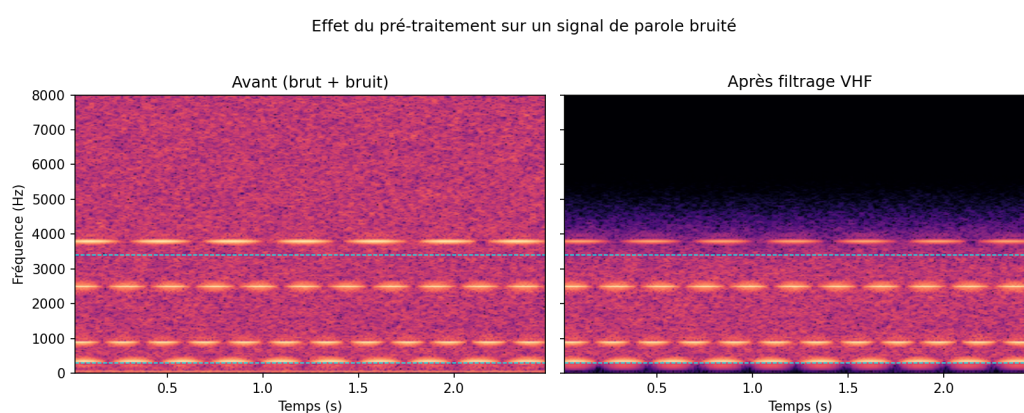


Figure 2 – Effet de la degradation VHF sur un signal de parole : spectre avant (gauche) et apres filtrage passe-bande (droite). Le contenu hors bande 300–3400 Hz est atténue, reproduisant la coloration d'un canal radio. La meme chaine est appliquee a la voix synthetisee.

Contraintes techniques et choix d'ingenierie

L'integration de XTTS dans l'environnement logiciel du cluster a necessite plusieurs ajustements. Le paquet `coqui-tts` a ete ajoute a l'environnement existant (*whisper-ate*) plutot que dans un environnement separe, afin de reutiliser la meme version de PyTorch et de `transformers` : ce choix evite une seconde installation de PyTorch, tient dans le quota disque de 20 Go et permet de servir les trois modeles (Whisper, Mistral, XTTS) depuis un unique processus. Comme XTTS importe un symbole (`isin_mps_friendly`) supprime dans `transformers` 5.x, ce symbole est restaure par un correctif applique avant l'import de la bibliotheque, sans retrograder `transformers` (qui doit rester en version 5.9 pour Whisper et Mistral). Enfin, l'encodeur de locuteur de XTTS declenche un bug cuFFT de `torchaudio` sur le GPU GH200 ; la synthese est donc executee sur CPU, ce qui la rend fiable mais non temps reel.

2.6 La boucle vocale complete

L'assemblage est valide par le module `voice_exchange.py`, qui rejoue un echange radio complet. Pour chaque instruction, le controleur parle (synthese + VHF), l'audio est transcrit puis interprete, BlueSky execute l'ordre, et l'aeronef collationne dans sa voix clonee (synthese + VHF). Le collationnement est ensuite re-transcrit pour verifier son intelligibilite, et les signaux sont

assembles en fichiers d'échange et en une bande son de session. La boucle complete *voix* → *Whisper* → *Mistral+RAG* → *BlueSky* → *readback vocal* a ete rejouee et verifiee le 2026-06-11 (descente FL280 → FL180 executee puis collationnee), comme l'illustre la figure 3.



Figure 3 – Application en mode ROMEO : apres une clairance vocale, l'aeronef AFR300 est etabli au FL180 et collationne dans sa voix clonee. La transcription de l'instruction apparait sous le bouton *push-to-talk*.

2.7 Resultats et mesures

Resultats cles

- **Latence de synthese** : 3,9 s par collationnement, facteur temps reel 0,82 (XTTS sur CPU, $n = 3$).
- **WER de re-transcription** de la voix synthetisee : 22 a 24 % (mesure sur la boucle S6/S8).
- **Boucle vocale complete** rejouee et verifiee de bout en bout (2026-06-11).
- **Trois voix de reference** clonees, attribuees a des aeronefs distincts.

Analyse. La re-transcription de la voix synthetisee atteint un WER de 22 a 24 %, contre 6,7 % pour une voix humaine réelle (validation de la semaine 4). Cet ecart est attendu : la boucle voix-clonee → Whisper cumule les erreurs de deux modeles successifs (la synthese et la reconnaissance), et la voix synthetique presente des caracteristiques hors du domaine d'entrainement du module de reconnaissance. Ce taux confirme neanmoins que le collationnement reste *intelligible* et exploitable pour un retour radio dans un contexte d'entrainement. La synthese reste par ailleurs stochastique : sur un meme texte, une generation propre et une generation plus degradee ont ete observees. Avec un facteur temps reel de l'ordre de 0,82 sur CPU, la synthese n'autorise pas un flux continu en temps reel, mais reste compatible avec le rythme d'une frequence réelle (un echange toutes les dizaines de secondes).

3 Semaine 7 : Integration – la requete unifiee

Objectif de la semaine

Construire une requete unifiee prenant en entree la transcription, les regles du RAG et une situation spatiale fixe, et produisant en sortie un JSON propre et exploitable decrivant l'action a mener (`{callsign, action, value, [wpt]}`).

3.1 Les trois entrees de la requete

La requete unifiee agrege trois sources d'information complementaires :

- **La transcription** : le texte de la clairance, issu du module de reconnaissance vocale (semaine 4) ou saisi directement.
- **Les regles du RAG** : les fiches de phraseologie les plus pertinentes pour cette clairance, recuperees par recherche vectorielle (semaine 5, $k = 4$).
- **La situation spatiale fixe** : une representation de l'espace aerien (le graphe de secteur), qui fournit au LLM la liste des points connus et la topologie, et qui sert ensuite a valider les ordres de routage.

3.2 La situation spatiale fixe : le graphe de secteur

La situation spatiale est decrite par un graphe statique du secteur, charge depuis `secteur_graphe.json`. Il comporte 7 points (de type *entry*, *fix* ou *exit*), 6 segments orientes avec leur distance, et une separation imposee de 5 NM (figure 4). La classe `SectorGraph` expose la liste des fixes, le voisinage, le plus court chemin (Dijkstra sur les distances) et un resume textuel compact de la topologie. C'est ce resume qui est injecte dans la requete pour donner au LLM une conscience spatiale, sans alourdir le *prompt*.

Listing 3 – Resume textuel de la topologie injecte dans la requete (`topology_text()`).

```
Sector (separation 5.0 NM).
Fixes: ENTRY_W(entry), BALMO(fix), CROSS(fix), DELTA(fix), EXIT_E(exit), ENTRY_S(entry),
       NORTH(exit).
Segments: ENTRY_W-BALMO 25.0NM; BALMO-CROSS 25.5NM; CROSS-DELTA 25.5NM; CROSS-ENTRY_S 25.0NM;
          CROSS-NORTH 25.0NM; DELTA-EXIT_E 25.0NM.
```

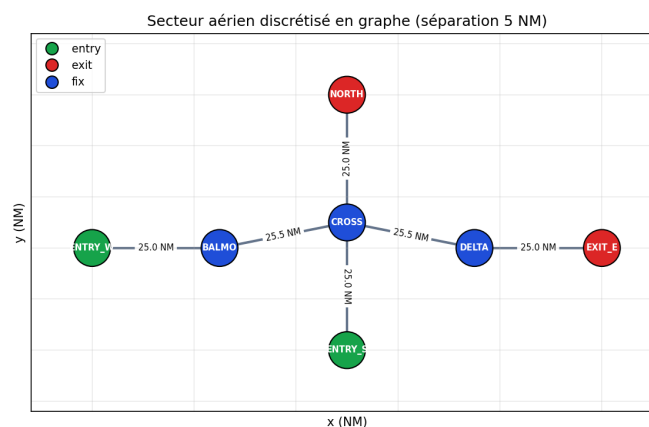


Figure 4 – Situation spatiale fixe : le graphe du secteur (points d'entree, fixes, sorties ; segments et distances). Il fournit le contexte spatial de la requete et valide les ordres de routage. A ce stade, cette topologie est statique (sans etat de trafic).

3.3 Construction de la requete unifiee

La fonction `build_messages` assemble les trois entrees en un unique message adresse au LLM, precede d'un *prompt* systeme strict qui impose une sortie au format JSON et seulement cela. Les regles recuperees, la topologie du secteur, les indices d'entites (indicatif, intentions) et la transcription sont concatenes dans un ordre fixe.

Listing 4 – Assemblage de la requete unifiee (extrait de `atc_llm.py`).

```
rules = "\n".join(f"- {d['title']}: {d['text']}" for d, _ in retrieved)
sector = f"Sector graph: {graph.topology_text()}\n\n"
hint = f"callsign={ner_callsign}; intentions_detectees={ner_intentions}"
user = (f"Applicable ICAO phraseology rules:\n{rules}\n\n"
        f"{sector}"
        f"NER hints: {hint}\n\n"
        f"Controller transcription: \"{transcription}\"\n\n"
        "Return ONLY the JSON array of orders.")
messages = [{"role": "system", "content": SYSTEM}, {"role": "user", "content": user}]
```

Sur un exemple, la requete assemblee et la sortie attendue prennent la forme suivante :

Listing 5 – Requete unifiee assemblee (abregee) et JSON produit par le LLM.

```
Applicable ICAO phraseology rules:
- Instruction de cap (heading): ... action HDG, value = cap en degres ...

Sector graph: Sector (separation 5.0 NM). Fixes: ENTRY_W(entry), BALMO(fix), ...

NER hints: callsign=AFR1234; intentions_detectees=['turn']

Controller transcription: "air france one two three four turn right heading two seven zero"

Return ONLY the JSON array of orders.
--> [{"callsign": "AFR1234", "action": "HDG", "value": 270}]
```

3.4 Orchestration : de la transcription au JSON

La fonction `interpret()` enchaîne les étapes de la requete unifiee : extraction des entites (NER) pour l'indicatif et les intentions, recuperation des $k = 4$ fiches pertinentes, assemblage de la requete, generation par le LLM (Mistral 7B), puis analyse tolerante du JSON et validation. La sortie est une liste d'ordres structures, chacun associe a sa traduction executable ou a un motif de rejet.

3.5 Garantir un JSON propre et exploitable

L'integration ne se limite pas a produire du JSON : elle doit produire du JSON *exploitable sans risque*. Quatre verifications deterministes s'appliquent en aval de la generation.

1. **Schema et bornes** : chaque ordre est traduit par `json_to_trafscript`; toute valeur hors bornes (HDG 0–360, ALT 0–45000, SPD 0–350) ou action inconnue est rejete.
2. **Coherence spatiale** : un ordre de routage (ADDWPT) doit cibler un fix *connu* du secteur, sinon il est rejete ou normalise vers le fix correct.
3. **Coherence semantique** : si la clairance dit “descend” (sans “climb”) mais que l'ordre vise un niveau *au-dessus* du niveau actuel de l'aeronef, l'ordre est juge incoherent (transcription tronquee ou hallucination) et rejete plutot qu'execute; le critere est symetrique pour “climb”.
4. **Realisme radio** : un ordre adresse a un indicatif absent du radar ne produit aucune reponse et n'est pas transmis au simulateur.

Contraintes techniques et choix d'ingenierie

A ce stade de l'integration, la situation spatiale fournie au LLM est volontairement *statique* : seule la topologie du secteur est injectee, pas encore l'etat de trafic (positions et niveaux des aeronefs), qui fera l'objet de l'etape suivante. La topologie est transmise sous forme d'un resume textuel compact plutot que du JSON complet du graphe, afin de limiter la taille de la requete et de ne pas diluer l'instruction. Le *prompt* systeme reste strict pour que l'ajout du bloc spatial ne pousse pas le modele a commenter sa reponse : seule la liste JSON est demandee, et une analyse tolerante recupere le tableau meme si le modele ajoute du texte. La verification de coherence semantique requiert l'altitude courante de l'aeronef ; elle s'applique donc au moment de l'execution, lorsque cet etat est disponible.

3.6 Resultats et mesures

Resultats cles

- **Requete unifiee** operationnelle : transcription, regles RAG et situation spatiale fixe combinees en une seule passe vers le LLM.
- **JSON exploitable** produit sur 40 transcriptions reelles : 100 %.
- **Validation de routage** : un ADDWPT vers un fix connu (BALMO) est accepte ; vers un point inconnu (tango bravo), il est rejete.
- **Garde-fou semantique** : une clairance tronquee de type "descend flight level one" qui ferait monter l'aeronef est detectee et rejete.

Analyse. La requete unifiee atteint son objectif : sur les transcriptions reelles evaluees, la sortie est systematiquement un JSON analysable, et les quatre couches de validation eliminant les ordres dangereux, mal route ou semantiquement incoherents avant toute execution. L'apport de l'integration ne tient pas seulement a l'ajout du contexte spatial, mais aussi a la *convergence* des verifications en un point unique : un meme ordre est confronte aux bornes physiques, a la topologie du secteur et a la coherence de l'instruction. La principale limite a ce stade est que la conscience spatiale du LLM reste statique : il connait la geometrie du secteur mais pas la position courante des aeronefs. C'est cette derniere information qui permettra, ensuite, des decisions reellement situees (anticipation de conflit, routage tenant compte du trafic).

4 Bilan et transition

Les semaines 6 et 7 marquent deux progressions complementaires. La semaine 6 complete la boucle de dialogue par un module de restitution vocale : le clonage *zero-shot* XTTS-v2 genere a la volee une voix distincte par aeronef sans entrainement par locuteur, et la degradation VHF reutilisee de la semaine 2 assure a la fois le realisme radio et la coherence de domaine avec le module de reconnaissance. La semaine 7 etablit le socle de l'integration : une requete unifiee combine la transcription, les regles du RAG et une situation spatiale fixe, et produit un JSON propre et exploitable decrivant l'action a mener, fiabilise par quatre verifications deterministes appliquees en aval du LLM (bornes, coherence spatiale, coherence semantique, presence au radar).

Prochaines etapes. L'etape directe consiste a remplacer la situation spatiale *fixe* par l'etat de trafic *courant* (positions et niveaux des aeronefs), afin de passer d'une conscience spatiale statique a une conscience situee. Suivront la consolidation de la boucle temps reel de bout en bout et une evaluation quantitative sur des scenarios complets.